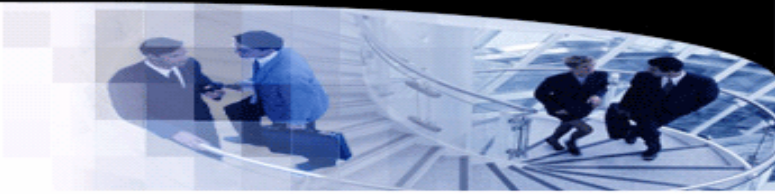
武漢大學
WUHAN UNIVERSITY

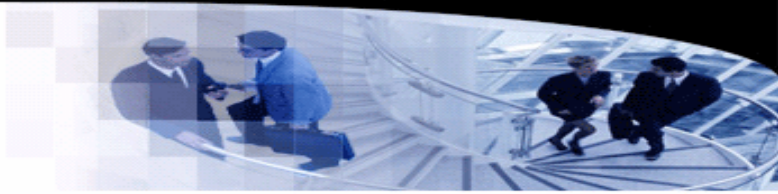


计算机科学经典理论与方法

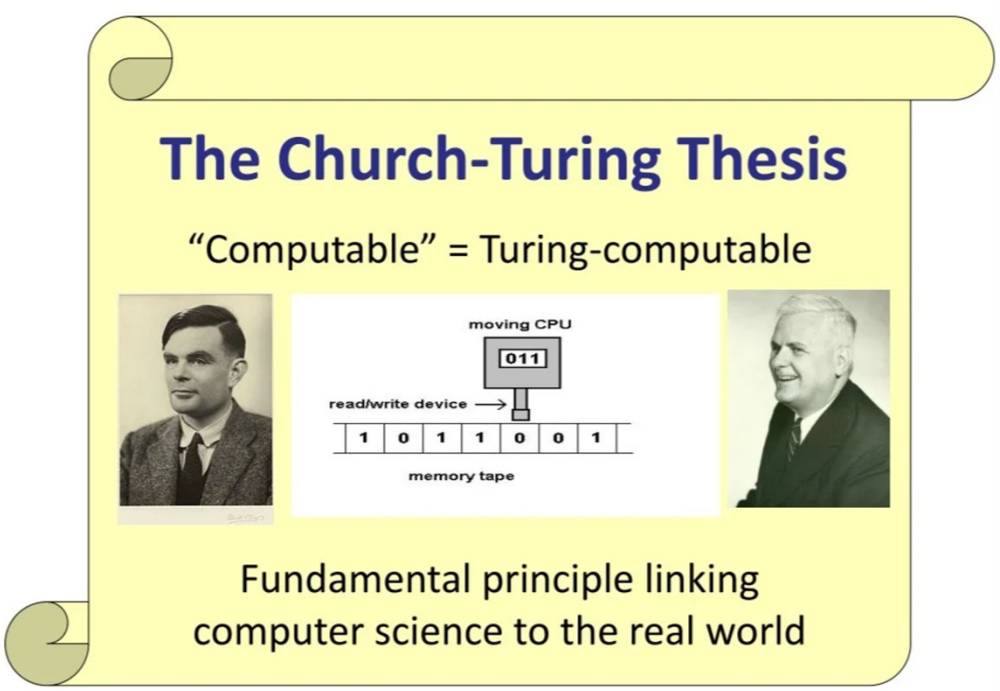
Lecture 1. Curry-Howard Correspondence

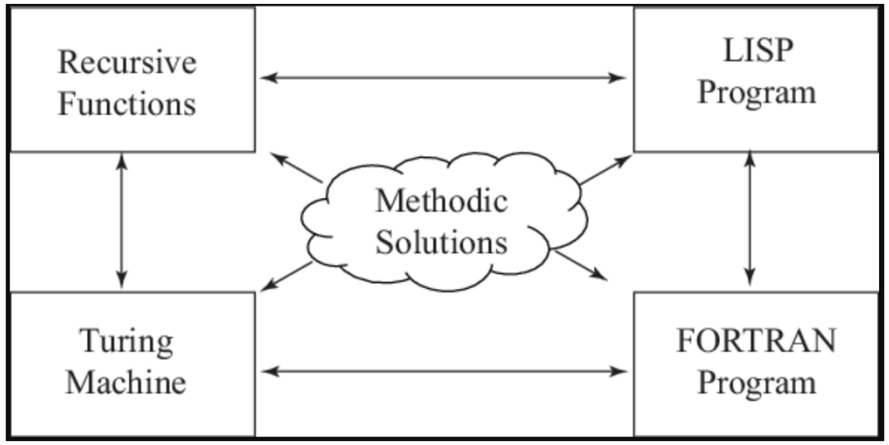
袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

1. 引言

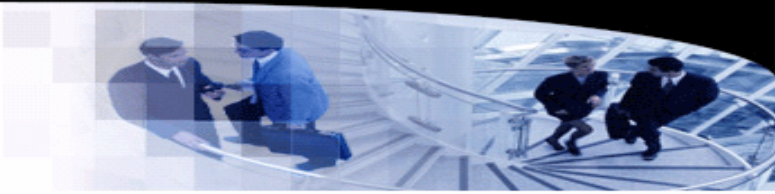


- 计算机科学中有很多相互联系的问题/理论/模型/方法
  - Church-Turing thesis

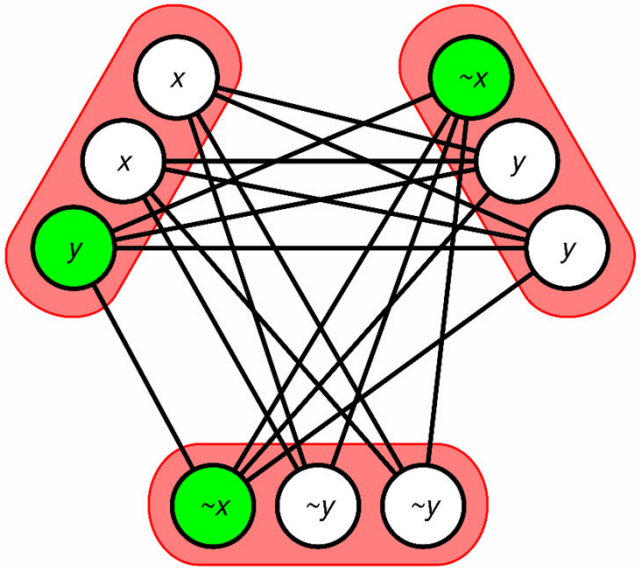




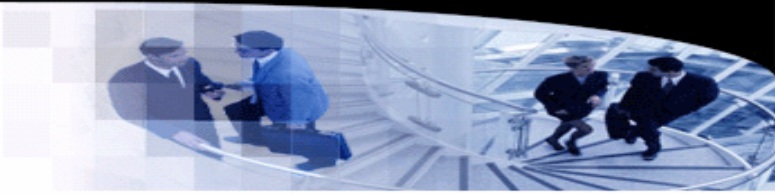
1. 引言



- 计算机科学中有很多相互联系的问题/理论/模型/方法
  - 3-SAT reduced to clique problem
    - (x ∨ x ∨ y) ∧ (¬x ∨ ¬y ∨ ¬y) ∧ (¬x ∨ y ∨ y) reduced to a clique problem.
    - The green vertices form a 3-clique and correspond to the satisfying assignment x=FALSE, y=TRUE.

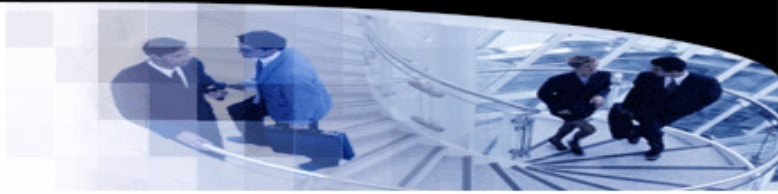


2. Curry-Howard Correspondence



- 简介
  - 由Haskell Curry（数学家）与William Alvin Howard（逻辑学家）发现
  - 揭示了计算机程序与数学证明之间的关系
- Correspondence
  - 类型（types）与逻辑公式（propositions）对应
  - 程序（programs）与逻辑证明（proofs）对应
  - 求值（evaluation）与证明化简（simplification）对应
  - A well-typed program is a proof.
  - Running a program is to simplifying a proof.
- 可以扩展到范畴论
  - Curry–Howard–Lambek correspondence

2. Curry-Howard Correspondence



- λ-calculus

→ *(typed)* — *Based on λ (5-3)*

Syntax

t	::=		*terms:*
		x	*variable*
		λx:T.t	*abstraction*
		t t	*application*
v	::=		*values:*
		λx:T.t	*abstraction value*
T	::=		*types:*
		T→T	*type of functions*
Γ	::=		*contexts:*
		∅	*empty context*
		Γ, x:T	*term variable binding*

Evaluation $\boxed{t \longrightarrow t'}$

$$\frac{t_1 \longrightarrow t_1'}{t_1\ t_2 \longrightarrow t_1'\ t_2} \quad \text{(E-App1)}$$

$$\frac{t_2 \longrightarrow t_2'}{v_1\ t_2 \longrightarrow v_1\ t_2'} \quad \text{(E-App2)}$$

$$(\lambda x:T_{11}.t_{12})\ v_2 \longrightarrow [x \mapsto v_2]t_{12} \quad \text{(E-AppAbs)}$$

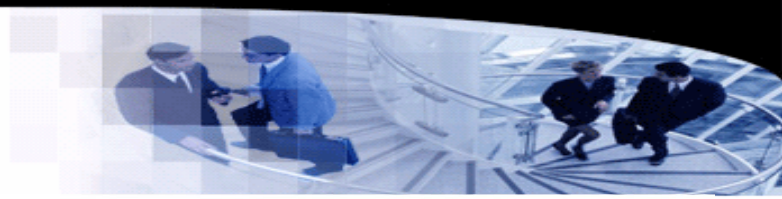
Typing $\boxed{\Gamma \vdash t : T}$

$$\frac{x:T \in \Gamma}{\Gamma \vdash x : T} \quad \text{(T-Var)}$$

$$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1.t_2 : T_1 \to T_2} \quad \text{(T-Abs)}$$

$$\frac{\Gamma \vdash t_1 : T_{11} \to T_{12} \qquad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1\ t_2 : T_{12}} \quad \text{(T-App)}$$

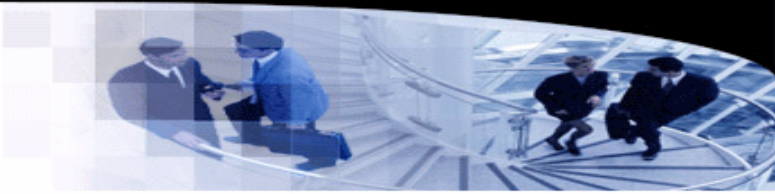
2. Curry-Howard Correspondence



- 逻辑与证明
  - 语法
    - $\phi ::= \top \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \to \phi)$
  - 自然推演法（Natural deduction）
    - 由一组推理规则（inference rules）组成
    - 例：And-introduction rule
      - $\dfrac{\phi \quad \psi}{\phi \wedge \psi} \wedge i$
    - 例：And-elimination rules
      - $\dfrac{\phi \wedge \psi}{\phi} \wedge e_1$
      - $\dfrac{\phi \wedge \psi}{\psi} \wedge e_2$

	introduction	*elimination*
$\wedge$	$\dfrac{\phi \quad \psi}{\phi \wedge \psi}\,\wedge\text{i}$	$\dfrac{\phi \wedge \psi}{\phi}\,\wedge\text{e}_1$ $\quad$ $\dfrac{\phi \wedge \psi}{\psi}\,\wedge\text{e}_2$
$\vee$	$\dfrac{\phi}{\phi \vee \psi}\,\vee\text{i}_1$ $\quad$ $\dfrac{\psi}{\phi \vee \psi}\,\vee\text{i}_2$	$\dfrac{\phi \vee \psi \quad \begin{array}{c}[\phi]\\ \vdots\\ \chi\end{array} \quad \begin{array}{c}[\psi]\\ \vdots\\ \chi\end{array}}{\chi}\,\vee\text{e}$
$\to$	$\dfrac{\begin{array}{c}[\phi]\\ \vdots\\ \psi\end{array}}{\phi \to \psi}\,\to\text{i}$	$\dfrac{\phi \quad \phi \to \psi}{\psi}\,\to\text{e}$
$\neg$	$\dfrac{\begin{array}{c}[\phi]\\ \vdots\\ \bot\end{array}}{\neg\phi}\,\neg\text{i}$	$\dfrac{\phi \quad \neg\phi}{\bot}\,\neg\text{e}$
$\bot$	(no introduction rule for $\bot$)	$\dfrac{\bot}{\phi}\,\bot\text{e}$
$\neg\neg$		$\dfrac{\neg\neg\phi}{\phi}\,\neg\neg\text{e}$

2. Curry-Howard Correspondence

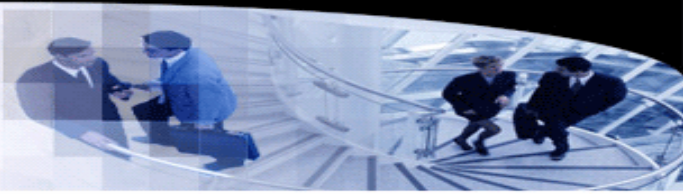


- 逻辑与证明
  - 证明（Proof）
    - 由前提（premise）到结论（conclusion）的推理过程
    - $\phi \vdash \psi$
    - 每一步推理使用一条推理规则
  - 例:

Show that $\neg p \vee q \vdash p \rightarrow q$ is valid.

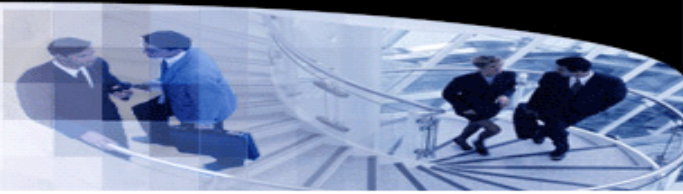
1	$\neg p \vee q$	premise
2	$\neg p$	assumption
3	p	assumption
4	$\bot$	$\neg\, e$ 3, 2
5	q	$\bot\, e$ 4
6	$p \rightarrow q$	$\rightarrow i$ 3-5
7	$p \rightarrow q$	$\vee\, e$ 1, 2-6
	q	assumption
	p	assumption
	q	copy 2
	$p \rightarrow q$	$\rightarrow i$ 3-4

3. 证据计算(Computing with Evidence)



- 重新思考类型与值
  - 值的类型
    - “123: int”被读为“123具有int类型”
  - 类型的证据
    - “123: int”被读为“123是int类型的证据”
    - 一个类型是一组值的集合
    - 123: int意味着int类型不为空，而123就是不为空的证据
  - OCaml中的空类型
    - type empty = |

3. 证据计算(Computing with Evidence)

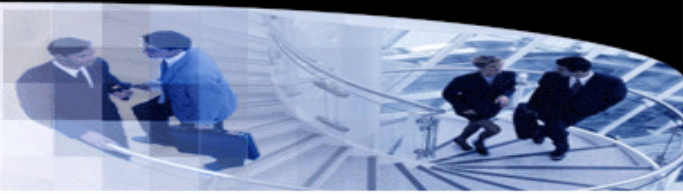


- 类型推理
  - 例:

```
# let pair x y = (x, y);;
val pair : 'a -> 'b -> 'a * 'b = <fun>
# let fst (x, y) = x;;
val fst : 'a * 'b -> 'a = <fun>
# let snd (x, y) = y;;
val snd : 'a * 'b -> 'b = <fun>
#
```

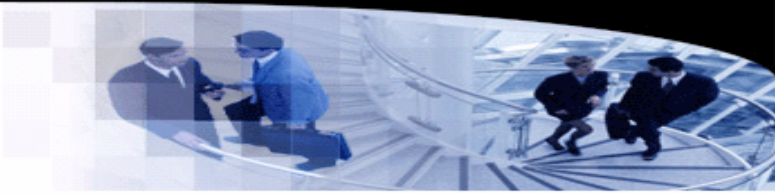
  - pair可以被看作一个函数，接收一个`a的证据和一个`b的证据，得到一个`a * `b的证据
  - pair可以看作一个论述：如果有一个`a的证据，那么，如果有一个`b的证据，可以得到`a和`b的证据
  - $A \to B \to (A \wedge B)$

3. 证据计算(Computing with Evidence)



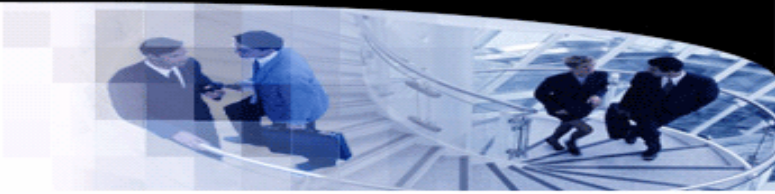
- 类型推理
  - 如果把`a替换为A，`b替换为B，*替换为∧，那么类型证据就变成了等价的逻辑公式(formula\proposition)
  - val pair : 'a -> 'b -> 'a * ‘b
    - $A \to B \to A \wedge B$
  - val fst : 'a * 'b -> ‘a
    - $A \wedge B \to A$
  - val snd : 'a * 'b -> ‘b
    - $A \wedge B \to B$

4. 类型与命题



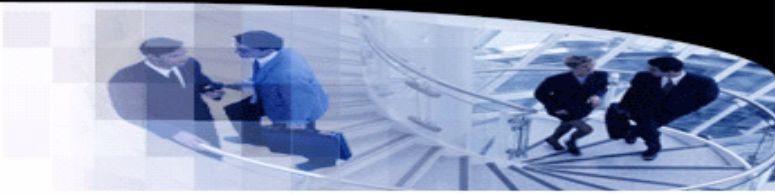
- 原子命题
  - 原子命题p可以成立或者不成立
  - T和⊥是两个特殊的原子命题
  - T永远成立
  - ⊥永远不成立

4. 类型与命题



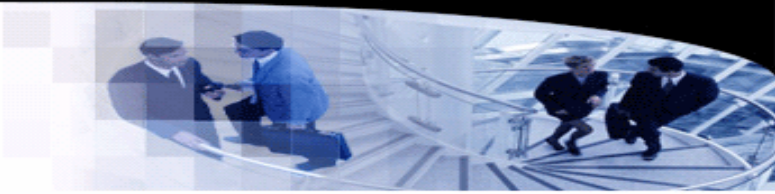
- Connectives
  - ∧, conjunction
    - 与乘积类型”*”对应
    - 一个值具有a * b类型的话，就意味着a和b类型都有证据
  - →, implication
    - 与函数类型→对应
    - $A \to B$: 如果A成立，那么B成立
    - fun $a \to b$: 将一个a的值转换为b的值，也就是说，如果类型a有证据，那么类型b就有证据

4. 类型与命题



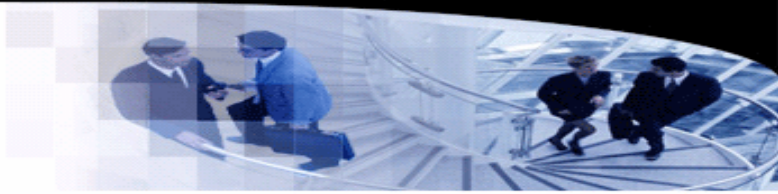
- Connectives
  - V, disjunction
    - $A \vee B$成立，当且仅当A或B中的一个命题成立
    - 与OCaml的variant type对应：
      - type ('a, 'b) disj = Left of 'a | Right of ‘b
    - 类型disj如果有一个值v，那么，v只能是以下两种情况之一：
      - 由Left构造（意味着v是'a的证据）
      - 由Right构造（意味着v是'b的证据）

4. 类型与命题



- ⊤与⊥
  - ⊤
    - ⊤永远成立
    - 对应的类型应该永远有证据
    - OCaml的unit类型只有一个值()
  - ⊥
    - ⊥永远不成立
    - 对应的类型应该为空
    - type empty = |
- Connectives
  - ¬, negation
    - ¬A等价于$A \rightarrow \bot$

5. 程序与证明



- Typing rules与deduction rules
  - 忽略变量，只考虑类型
  - 等价的推演规则/定理：
    - $\dfrac{}{p \vdash p}$
    - $\dfrac{\Gamma, p_1 \vdash p_2}{\Gamma \vdash p_1 \to p_2}$
    - $\dfrac{\Gamma \vdash p_1 \to p_2 \qquad \Gamma \vdash p_1}{\Gamma \vdash p_2}$

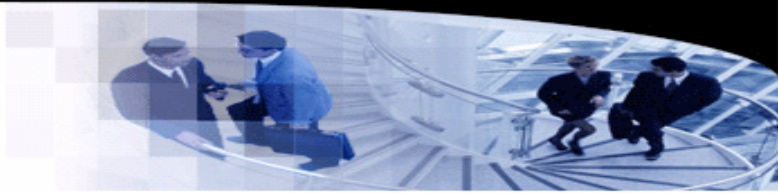
Typing $\quad \boxed{\Gamma \vdash t : T}$

$$\frac{x{:}T \in \Gamma}{\Gamma \vdash x : T} \quad \text{(T-VAR)}$$

$$\frac{\Gamma, x{:}T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x{:}T_1 . t_2 : T_1 \to T_2} \quad \text{(T-ABS)}$$

$$\frac{\Gamma \vdash t_1 : T_{11} \to T_{12} \qquad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1\ t_2 : T_{12}} \quad \text{(T-APP)}$$

5. 程序与证明

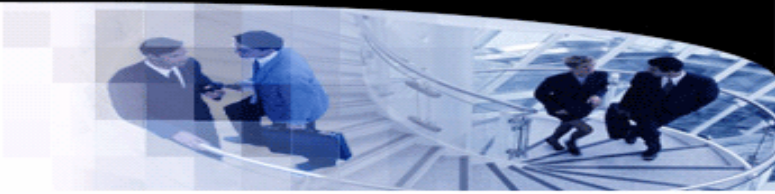


- 程序
  - 试推理程序的`fun p -> (snd p, fst p)`的类型

$$\dfrac{\dfrac{\dfrac{}{\{p{:}a*b\}\vdash\{p{:}a*b\}}}{\{p{:}a*b\}\vdash snd\ p{:}\ b}\quad\dfrac{\dfrac{}{\{p{:}a*b\}\vdash\{p{:}a*b\}}}{\{p{:}a*b\}\vdash fst\ p{:}\ a}}{\{p{:}a*b\}\vdash(snd\ p, fst\ p){:}\ b*a}$$

$$\{\quad\}\vdash fun\ p{-}{>}(snd\ p, fst\ p){:}\ a*b\to b*a$$

5. 程序与证明

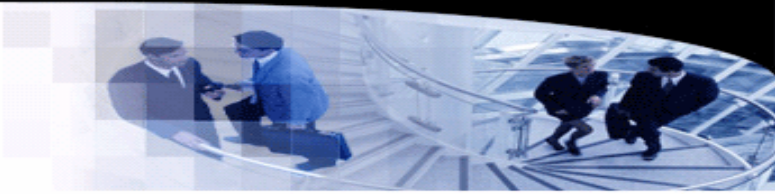


- 证明
  - 试证明$\vdash a \wedge b \rightarrow b \wedge a$

$$\dfrac{\dfrac{\dfrac{}{a \wedge b \vdash a \wedge b}}{a \wedge b \vdash b} \quad \dfrac{\dfrac{}{a \wedge b \vdash a \wedge b}}{a \wedge b \vdash a}}{a \wedge b \vdash b \wedge a}$$

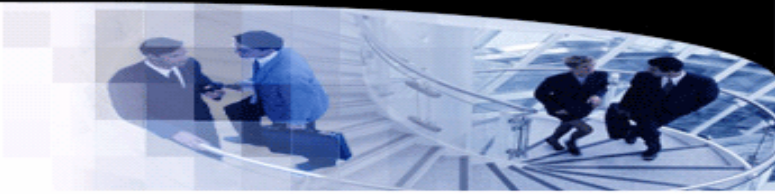
$$\vdash a \wedge b \rightarrow b \wedge a$$

6. 求值与化简



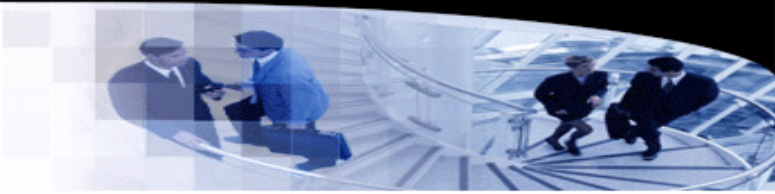
- 求值（Evaluation）
  - 将$fst\ (a,b)$求值为a，则等价于推断a的类型
  - $$\frac{}{\{a{:}t,\ b{:}t'\}\vdash a{:}t}$$
  - 对应的证明：
  - $$\frac{}{t,t'\vdash t}$$

6. 求值与化简



- 求值（Evaluation）
  - 考虑求值规则：$fst\ (a,b) \longrightarrow a$。假设a和b的类型已知，试推断fst (a, b)的类型
  - $$\frac{\dfrac{\overline{\{a{:}t,\ b{:}t'\}\vdash a{:}t}\quad\overline{\{a{:}t,\ b{:}t'\}\vdash b{:}t'}}{\{a{:}t,\ b{:}t'\}\vdash (a,b){:}t*t'}}{\{a{:}t,\ b{:}t'\}\vdash fst\ (a,b){:}t}$$
  - 对应的证明：
  - $$\frac{\dfrac{\overline{t,t'\vdash t}\quad\overline{t,t'\vdash t'}}{t,t'\vdash t\wedge t'}}{t,t'\vdash t}$$

7. 总结



- “Functional Programming in OCaml” - Spring 2021 Edition
  - The Curry-Howard correspondence shows that logic and computation are fundamentally linked in a deep and maybe even mysterious way. The basic building blocks of logic (propositions, proofs) turn out to correspond to the basic building blocks of computation (types, functional programs). Computation itself, the evaluation or simplification of expressions, turns out to correspond to simplification of proofs. The very task that computers do therefore is the same task that humans do in trying to present a proof in the best possible way.
  - Computation is thus intrinsically linked to reasoning. And functional programming is a fundamental part of human inquiry.
- 参考
  - Philip Wadler. 2015. Propositions as types. Commun. ACM 58, 12 (December 2015), 75–84. https://doi.org/10.1145/2699407